



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

EXPERIMENT 4

Aim: To build a data pipeline that applies multiple transformation to a dataset creating a repeatable process using Apache Airflow.

Theory:

Apache Airflow is an open-source workflow automation tool designed to orchestrate, schedule, and monitor workflows (DAGs - Directed Acyclic Graphs). It allows users to define workflows as Python code, making them dynamic and scalable. Airflow ensures efficient execution of complex data workflows by managing dependencies, retries, and parallel executions.

Key Features of Apache Airflow

- Scalability: Can handle large-scale workflows efficiently.
- Extensibility: Integrates with cloud services like AWS, GCP, and Azure.
- UI & Monitoring: Provides a web-based interface for managing DAGs.
- Task Dependencies: Automatically resolves task execution order.
- Retry Mechanism: Retries failed tasks to ensure pipeline reliability.
- Parallel Execution: Supports multiple task execution using Celery Executor.

Implementation of a Pipeline in Apache Airflow

A pipeline in Apache Airflow is defined using DAGs. A DAG is a collection of tasks, executed in a specified order based on dependencies.

Steps to Implement a Pipeline:

1. Install Apache Airflow
 - o Install Airflow using pip and set up a virtual environment.
 - Initialize the database and start the webserver & scheduler.
2. Define a DAG (Workflow Pipeline)
 - o Create a DAG file in `~/airflow/dags/`.
 - o Define tasks using operators (PythonOperator, BashOperator, etc.).
 - Set task dependencies (`task_1 >> task_2` for sequential execution).
3. Start Airflow Services
 - o Run `airflow webserver --port 8080` to launch the UI.



Name: Gargi Sukhatankar

Roll No: B027

SAP ID: 60019220125

Branch: CSE(ICB)

- Run airflow scheduler to start executing workflows.
4. Trigger and Monitor the DAG o Access <http://localhost:8080>, enable the DAG, and trigger execution.
- Monitor tasks using the UI (Graph View, Tree View, Log

Screenshots:

1) Apache installation:

```
student@MUM0922CPU0070:~$ python3 -m venv airflow_venv
source airflow_venv/bin/activate
The virtual environment was not created successfully because ensurepip is not
available. On Debian/Ubuntu systems, you need to install the python3-venv
package using the following command.

apt install python3.8-venv

You may need to use sudo with that command. After installing the python3-venv
package, recreate your virtual environment.

Failing command: ['/home/student/airflow_venv/bin/python3', '-Im', 'ensurepip', '--upgrade', '--default-pip']

student@MUM0922CPU0070:~$ source airflow_venv/bin/activate
-bash: airflow_venv/bin/activate: No such file or directory
student@MUM0922CPU0070:~$ ^V^C
student@MUM0922CPU0070:~$ sudo apt update && sudo apt upgrade -y
Get:1 http://security.ubuntu.com/ubuntu focal-security InRelease [128 kB]
Hit:2 http://archive.ubuntu.com/ubuntu focal InRelease
Get:3 http://archive.ubuntu.com/ubuntu focal-updates InRelease [128 kB]
Get:4 http://security.ubuntu.com/ubuntu focal-security/main amd64 Packages [3421 kB]
```

```
(airflow_venv) student@MUM0922CPU0070:~$ airflow db init
/home/student/airflow_venv/lib/python3.8/site-packages/airflow/cli/commands/db_command.py:43 DeprecationWarning: 'db init' is deprecated. Use 'db
to create the default connections
DB: sqlite:///home/student/airflow/airflow.db
[1025-03-04T11:24:22.343+0530] {migration.py:213} INFO - Context impl SQLiteImpl.
[1025-03-04T11:24:22.343+0530] {migration.py:216} INFO - Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Context impl SQLiteImpl.
INFO [alembic.runtime.migration] Will assume non-transactional DDL.
INFO [alembic.runtime.migration] Running stamp_revision -> 405de8318b3a
WARNI [airflow.models.crypto] empty cryptography key - values will not be stored encrypted.
Initialization done
(airflow_venv) student@MUM0922CPU0070:~$ airflow users create \
> --username admin \
> --password admin123 \
> --firstname Admin \
> --lastname User \
> --role Admin \
> --email admin@example.com
/home/student/airflow_venv/lib/python3.8/site-packages/flask_limiter/extension.py:336 UserWarning: Using the in-memory storage for tracking rate l
ee: https://flask-limiter.readthedocs.io#configuring-a-storage-backend for documentation about configuring the storage backend.
[1025-03-04T11:24:37.090+0530] {manager.py:239} INFO - Inserted Role: %s
[1025-03-04T11:24:37.105+0530] {manager.py:239} INFO - Inserted Role: %s
[1025-03-04T11:24:37.107+0530] {manager.py:670} WARNING - No user yet created, use flask fab command to do it.
[1025-03-04T11:24:37.200+0530] {manager.py:499} INFO - Created Permission View: %s
[1025-03-04T11:24:37.214+0530] {manager.py:555} INFO - Added Permission %s to role %s
[1025-03-04T11:24:37.241+0530] {manager.py:499} INFO - Created Permission View: %s
```




Name: Gargi Sukhatankar

Roll No: B027

--username admin \

--password admin123 \

--firstname Admin \

--lastname User \

--role Admin \

--email admin@example.com

SAP ID: 60019220125

Branch: CSE(ICB)

1. In the first terminal, start the Airflow web server:

```
airflow webserver --port 8080
```

2. In the second terminal, start the Airflow scheduler:

```
airflow scheduler
```

Apache airflow webserver

Pipeline implementation:

```
cd ~/airflow/dags
```

```
nano my_pipeline.py
```

```
from airflow import DAG
```

```
from airflow.operators.dummy import DummyOperator
```

```
from airflow.operators.python import PythonOperator
```

```
from datetime import datetime, timedelta
```

```
# Function for Python Task
```

```
def my_task():
```

```
print("Executing my custom task in Airflow")
```

```
# Define default arguments
```

```
default_args = {
```

```
"owner": "airflow",
```



**SHRI VILEPARLE KELAVANI MANDAL'S
DWARKADAS J. SANGHVI COLLEGE OF ENGINEERING**
(Autonomous College Affiliated to the University of Mumbai)
NAAC ACCREDITED with "A" GRADE (CGPA : 3.18)



Name: Gargi Sukhatankar
Roll No: B027

SAP ID: 60019220125
Branch: CSE(ICB)

my_pipeline_Vipul A simple Airflow pipeline

Schedule: 1 day, 0:00:00 | Next Run: 2025-03-04, 00:00:00

Grid | Graph | Calendar | Task Duration | Task Times | Landing Times | Gantt | Details | Code | Audit Log

04-03-2025 09:04:25 | 25 | All Run Types | All Run States | Clear Filters | Auto-refresh

Press shift + / for Shortcuts

Deferred | Failed | Queued | Removed | Restarting | Running | Scheduled | Skipped | Success | Up for Reschedule | Up for Retry | Upstream Failed | No Status

DAG Summary

Total Tasks	3
Empty Operators	2
PythonOperator	1

DAG Details

Dag id	my_pipeline_Vipul
Description	A simple Airflow pipeline
Fileloc	/home/student/airflow/dags/my_pipeline_Vipul.py
Has import errors	false
Has task concurrency limits	false
Is active	true
Is paused	true
Is subdag	false

example_trigger_target_dag airflow None 2025-03-03 00:00:00+00:00

example_weekday_branch_operator airflow @daily 2025-03-03 00:00:00+00:00

example_xcom airflow @once 2021-01-01 00:00:00+00:00

example_xcom_args airflow None

example_xcom_args_with_operators airflow None

latest_only airflow 4:00:00 2025-03-04 05:02:54.078582+00:00

latest_only_with_trigger airflow 4:00:00 2025-03-04 05:02:52.902129+00:00

my_pipeline airflow 1 day, 0:00:00 2025-03-04 00:00:00+00:00

my_pipeline_vipul airflow 1 day, 0:00:00 2025-03-04 00:00:00+00:00

tutorial airflow 1 day, 0:00:00 2025-03-03 09:02:28.429881+00:00

tutorial_dag airflow None

tutorial_taskflow_api airflow None

Showing 1-12 of 52 DAGs

Version: v2.7.2

"depends_on_past": False,
"start_date": datetime(2025, 3, 4),



Name: Gargi Sukhatankar

Roll No: B027

"retries": 1,

"retry_delay": timedelta(minutes=5),

}

Define the DAG

with DAG(

"my_pipeline",

default_args=default_args,

description="A simple Airflow pipeline",

schedule_interval=timedelta(days=1), # Runs daily

catchup=False,

) as dag:

start = DummyOperator(task_id="start")

task_1 = PythonOperator(

task_id="my_python_task",

python_callable=my_task,

)

end = DummyOperator(task_id="end")

Define Task Order

start >> task_1 >> end

Conclusion:

Thus we successfully built a data pipeline that applies multiple transformation to a dataset creating a repeatable process using Apache Airflow.

SAP ID: 60019220125

Branch: CSE(ICB)